



CAO PROJECT

INSTRUCTOR: MS. AMINAH AKHTAR

COURSE CODE: CS-252

SECTION : B

MINI RETAIL BILLING SYSTEM

Generates the invoice for the purchased items



MUHAMMAD MOAZZAM FAROOQUI CT-24068

HUSN UR REHMAN KHAN CT-24070

DANYAL ABBAS CT-24084



Project Title: Smart Retail Billing & Invoice Generation System using 8086 Assembly Language

PROJECT REPORT

1. SYSTEM OVERVIEW

PROBLEM STATEMENT:

Design a Mini Retail Billing System for a small shop that:

- Stores at least 5 items with names and prices in memory
- Allows customers to select items and enter quantities
- Computes item totals and grand totals
- Generates an invoice
- Implements a menu-driven, interactive interface

SYSTEM DESIGN:

The system is named "JZ MART" (Jump If Zero Mart) and operates through three main phases:

1. **INITIALIZATION:** Program displays a welcome message and item menu showing all 5 available items with prices.
2. **SHOPPING PHASE:** The user can select items, enter the quantity of the item they want to purchase, and then the system adds the amount to the running total. The user can also purchase multiple items, and the loop continues until the user chooses to check out.
3. **INVOICE PHASE:** When the user chooses to check out by pressing 6, an invoice is generated that shows the entire purchase, along with the discount and the total discounted amount if the discount is achieved.

ITEMS INVENTORY:

Item	Price (RS)
1. VEGETABLES	80
2. EGGS (DOZEN)	300
3. CANDIES	5
4. DAIRY PRODUCTS	250
5. FRUITS	150

DISCOUNT SLABS (TIERED):

- Purchase $\geq$ 10,000 RS: 50% discount
- Purchase $\geq$ 6,000 RS: 30% discount
- Purchase $\geq$ 4,000 RS: 20% discount
- Purchase $\geq$ 2,000 RS: 10% discount
- Purchase $<$ 2,000 RS: No discount

2. PROGRAM DESIGN

2.1 DATA STRUCTURES AND VARIABLES

Item names and prices are stored as strings in the **.DATA** section. The prices, quantity, and other messages are stored via arrays, and the variables are used to store the total of the purchases and formatting spaces.

Variables Definition:

```
047      ;Arrays
048      ;Price array <index 0=Veg, 1=Eggs, 2=Candy, 3=Dairy, 4=Fruits>
049      Prices      DW 80, 300, 5, 250, 150
050
051      ;Quantity array <parallel to Prices>
052      Counts      DW 0, 0, 0, 0, 0
053
054      ;Name pointer array <stores offsets of each Name_xx string>
055      NamePtrs      DW Name_Veg, Name_Eggs, Name_Candy, Name_Dairy, Name_Fruits
056
057      ;Prompt pointer array <stores offsets of each Prompt_xx string>
058      PromptPtrs    DW Prompt_Veg, Prompt_Eggs, Prompt_Candy, Prompt_Dairy, Prompt_Fruits
059
060      ;Variables
061      Grand_Total    DW 0
062      DigitCount     DW 0
```

- **Prices Array:** Stores the pricing of the items.
- **Quantity Array:** Stores the quantity of the items purchased.
- **NamePtr:** Stores the offset of each name.
- **PromptPtrs:** Stores the offset of each Prompt.
- **Grand Total:** Stores the total amount of the purchased items.
- **DigitCount:** Stores the required spaces for the formatted invoice and for printing.

Messages Strings:

Stores the messages in .data Segment to be accessible for the printing module.

```
0001 .MODEL SMALL
0002 .STACK 100H
0003
0004 .DATA
0005
0006 ;UI Messages
0007 Welcome_Msg DB 10,13,'WELCOME TO JUMP IF HUNGRY <JZ> MART$'
0008 Menu_Header DB 10,13,10,13,'----- ITEMS MENU -----$',10,13
0009 Item_Menu_1 DB 10,13,'1. VEGETABLES - 80 RS$'
0010 Item_Menu_2 DB 10,13,'2. EGGS <DOZEN> - 300 RS$'
0011 Item_Menu_3 DB 10,13,'3. CANDIES - 5 RS$'
0012 Item_Menu_4 DB 10,13,'4. DAIRY PRODUCTS - 250 RS$'
0013 Item_Menu_5 DB 10,13,'5. FRUITS - 150 RS$'
0014
0015 Status_Msg DB 10,13,10,13,'CURRENT TOTAL: RS $'
0016 Choice_Msg DB 10,13,'SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: $'
0017
0018 ;Quantity Prompts
0019 Prompt_Veg DB 10,13,'HOW MANY VEGETABLES DO YOU WANT TO PURCHASE? $'
0020 Prompt_Eggs DB 10,13,'HOW MANY DOZEN EGGS DO YOU WANT TO PURCHASE? $'
0021 Prompt_Candy DB 10,13,'HOW MANY CANDIES DO YOU WANT TO PURCHASE? $'
0022 Prompt_Dairy DB 10,13,'HOW MANY DAIRY PRODUCTS DO YOU WANT TO PURCHASE? $'
0023 Prompt_Fruits DB 10,13,'HOW MANY FRUITS DO YOU WANT TO PURCHASE? $'
0024
0025 ;Invoice Header
0026 Invoice_Title DB 10,13,10,13,'===== FINAL INVOICE =====$'
0027 Column_Header DB 10,13,'ITEM NAME QTY PRICE ITEM TOTAL$'
0028 Separator_Line DB 10,13,'-----$'
0029
0030 ;Item Name Strings
0031 Name_Veg DB 10,13,'Vegetables $'
0032 Name_Eggs DB 10,13,'Eggs $'
0033 Name_Candy DB 10,13,'Candy $'
0034 Name_Dairy DB 10,13,'Dairy $'
0035 Name_Fruits DB 10,13,'Fruits $'
0036
0037 ;Congratulations Messages
0038 Sub_Total_Msg DB 10,13,10,13,'SUBTOTAL: RS $'
0039 Congrats_10 DB 10,13,'CONGRATULATIONS! YOU GOT A 10% DISCOUNT!$'
0040 Congrats_20 DB 10,13,'CONGRATULATIONS! YOU GOT A 20% DISCOUNT!$'
0041 Congrats_30 DB 10,13,'CONGRATULATIONS! YOU GOT A 30% DISCOUNT!$'
0042 Congrats_50 DB 10,13,'CONGRATULATIONS! YOU GOT A 50% DISCOUNT!$'
0043 Discount_Prefix DB 10,13,'BONUS DISCOUNT: RS $'
0044 Grand_Total_Msg DB 10,13,'GRAND TOTAL: RS $'
0045 Exit_Msg DB 10,13,10,13,'THANK YOU FOR SHOPPING AT JZ MART!$'
```

2.2 MACROS

PRINT Macro (Lines 64-69):

prints string at label from DOS INT 21H/09H

```
064 ;Macros
065 PRINT MACRO MSG
066 LEA DX, MSG
067 MOV AH, 09H
068 INT 21H
069 ENDM
```

Purpose: *Simplify string output to avoid code repetition*

- **Function:** Loads the address of the message string into DX and helps to print the string.
- **Usage:** PRINT Welcome_Msg, PRINT Choice_Msg, etc. (36 uses total)
- **Benefit:** Reduces code redundancy

2.3 PROCEDURES AND THEIR ROLES

MAIN PROCEDURE (Lines 65-332):

- Initializes the data segment.
- Displays a welcome message and an item menu.
- Manages the main shopping loop.
- Processes user selections.
- Calls invoice generation when requested.
- Terminates program.

ITEM SELECTION LOOP (Lines 91 - 128):

- Displays the current running total.
- Prompts user for selection (1- 5 for items, 6 for invoice or terminate).
- Uses CMP and JE (jump if equal) to route to the appropriate handler.
- SI is used here to set an offset into arrays for accessibility (works like an index).
- Continues loop until user selects option 6.

```
091 ;-----
092 ; ITEM SELECTION LOOP
093 ;-----
094 ITEM_SELECTION_LOOP:
095     PRINT Status_Msg
096     MOV AX, Grand_Total
097     CALL PRINT_NUMBER
098     PRINT Choice_Msg
099
100     MOV AH, 01H
101     INT 21H
102     SUB AL, 48          ; Convert ASCII digit to numeric value
103
104     CMP AL, 6
105     JE GENERATE_INVOICE
106     CMP AL, 1
107     JE IDX_0
108     CMP AL, 2
109     JE IDX_1
110     CMP AL, 3
111     JE IDX_2
112     CMP AL, 4
113     JE IDX_3
114     CMP AL, 5
115     JE IDX_4
116     JMP ITEM_SELECTION_LOOP
117
118 |
119 IDX_0: MOV SI, 0
120       JMP BUY_ITEM
121 IDX_1: MOV SI, 2
122       JMP BUY_ITEM
123 IDX_2: MOV SI, 4
124       JMP BUY_ITEM
125 IDX_3: MOV SI, 6
126       JMP BUY_ITEM
127 IDX_4: MOV SI, 8
128       JMP BUY_ITEM
129
```

BUY ITEM PROCEDURE (Lines 125 - 143):

```
125 ; -----
126 ; BUY_ITEM PROCEDURE <Single handler replacing 5 separate ones>
127 ; SI = byte offset into Prices[], Counts[], NamePtrs[], PromptPtrs[]
128 ; -----
129 BUY_ITEM:
130     MOV BX, PromptPtrs[SI]
131     MOV DX, BX
132     MOV AH, 09H
133     INT 21H
134
135     CALL GET_QUANTITY
136
137     ADD Counts[SI], AX
138
139     MOV BX, Prices[SI]
140     MUL BX
141     ADD Grand_Total, AX
142
143     JMP ITEM_SELECTION_LOOP
```

- Displays the quantity string according to the item the user wishes to purchase.
- Call GET_QUANTITY to read input.
- Add quantity to the item's counter.
- Multiply the quantity by the item's price.
- Add it to Grand_Total.
- Redirected to selection loop.

GENERATE INVOICE PROCEDURE (Lines 145 - 198):

```
145 ; -----
146 ; GENERATE INVOICE PROCEDURE
147 ; Iterates through arrays using LOOP + SI index
148 ; -----
149 GENERATE_INVOICE:
150     MOV AX, 03H
151     INT 10H
152
153     PRINT Invoice_Title
154     PRINT Column_Header
155     PRINT Separator_Line
156
157     MOV SI, 0
158     MOV CX, 5
159
160 INVOICE_LOOP:
161     CMP Counts[SI], 0
162     JE SKIP_ITEM
163
164     ;Prints the item name
165     MOV BX, NamePtrs[SI]
166     MOV DX, BX
167     MOV AH, 09H
168     INT 21H
169
170     ;Prints the quantity
171     MOV AX, Counts[SI]
172     CALL PRINT_NUMBER
173     PUSH CX
174     MOV CX, 8
175     SUB CX, DigitCount
176     CALL PRINT_SPACES
177     POP CX
178
179     ;Prints the unit price
180     MOV AX, Prices[SI]
181     CALL PRINT_NUMBER
182     PUSH CX
183     MOV CX, 10
184     SUB CX, DigitCount
185     CALL PRINT_SPACES
186     POP CX
```

```

188      ;Calculates and print the item total
189      MOV AX, Counts[SI]
190      MOV BX, Prices[SI]
191      MUL BX
192      CALL PRINT_NUMBER
193
194  SKIP_ITEM:
195      ADD SI, 2                      ;Advance to next DW element
196      LOOP INVOICE_LOOP
197
198      PRINT Separator_Line

```

- Clears the screen before printing the invoice table.
- Prints the string messages from the .data segment.
- Checks if any item is not purchased, then its string is not printed.
- Prints the name of the item, quantity, price, and total amount of the item purchased.

FINANCIAL SUMMARY AND DISCOUNT LOGIC(Lines 200-234):

```

200  ;-----
201  ; FINANCIAL SUMMARY AND DISCOUNT LOGIC
202  ;-----
203      PRINT Sub_Total_Msg
204      MOV AX, Grand_Total
205      CALL PRINT_NUMBER
206
207      MOV AX, Grand_Total
208
209      ;Types of discount applied based on total amount of purchases
210      CMP AX, 10000
211      JAE APPLY_50
212      CMP AX, 6000
213      JAE APPLY_30
214      CMP AX, 4000
215      JAE APPLY_20
216      CMP AX, 2000
217      JAE APPLY_10
218      JMP DISPLAY_FINAL
219
220  APPLY_50:
221      PRINT Congrats_50
222      MOV BX, 50
223      JMP DO_MATH
224
225  APPLY_30:
226      PRINT Congrats_30
227      MOV BX, 30
228      JMP DO_MATH
229
230  APPLY_20:
231      PRINT Congrats_20
232      MOV BX, 20
233      JMP DO_MATH
234
235  APPLY_10:
236      PRINT Congrats_10
237      MOV BX, 10

```

- Implements the discount logic based on the criteria mentioned in the PDF.
- If a certain discount applied, the flow transferred there.
- Calculates the discounted amount and subtracts it from grand total. (lines 241 - 251)
- Prints the final amount, and after that prints an exit message.

HELPERS (Lines 259-352):

GET_QUANTITY (Lines 268-294):

```
259 ; =====
260 ; HELPER PROCEDURES
261 ; =====
262 ;
263 ;
264 ; GET_QUANTITY: Reads multi-digit integer from keyboard
265 ; Returns: integer value in AX
266 ; Algorithm: accumulate-and-shift (BX = BX*10 + digit)
267 ; -----
268 GET_QUANTITY PROC
269     PUSH BX
270     PUSH CX
271     MOV BX, 0
272     MOV CX, 10
273
274 GET_Q_LOOP:
275     MOV AH, 01H
276     INT 21H
277     CMP AL, 13
278     JE GET_Q_DONE
279     SUB AL, 48
280     MOV AH, 0
281     PUSH AX
282     MOV AX, BX
283     MUL CX
284     MOV BX, AX
285     POP AX
286     ADD BX, AX
287     JMP GET_Q_LOOP
288
289 GET_Q_DONE:
290     MOV AX, BX
291     POP CX
292     POP BX
293     RET
294 GET_QUANTITY ENDP
```

- The main purpose is to fetch the user input value for the quantity
- The get quantity loops decode the value entered into its ASCII key, then converts into numeric data, and then the digit is decoded through that.
- After getting the quantity, the get_quantity procedure ends.

PRINT_SPACES (Lines 296-314):

- Prints spaces for the alignment of quantity, price, and total price of every item.

```
296 ; -----
297 ; PRINT_SPACES: Prints CX spaces for column alignment
298 ; Receives: CX = number of spaces
299 ; -----
300 PRINT_SPACES PROC
301     CMP CX, 0
302     JLE PS_END
303     PUSH AX
304     PUSH DX
305     MOV AH, 02H
306     MOV DL, ' '
307 PS_L:
308     INT 21H
309     LOOP PS_L
310     POP DX
311     POP AX
312 PS_END:
313     RET
314 PRINT_SPACES ENDP
```


PRINT_NUMBER (Lines 316-352):

Purpose: Convert a 16-bit unsigned integer to ASCII decimal and display

Returns: Sets DigitCount variable (used for column alignment)

```
316 ;-----  
317 ; PRINT_NUMBER: Converts 16-bit unsigned int in AX to ASCII and displays  
318 ; Sets DigitCount for use by PRINT_SPACES alignment  
319 ; Algorithm: divide by 10, push remainders, pop in order  
320 ;-----  
321 PRINT_NUMBER PROC  
322     PUSH AX  
323     PUSH BX  
324     PUSH CX  
325     PUSH DX  
326     MOV DigitCount, 0  
327     MOV BX, 10  
328     MOV CX, 0  
329  
330 PN_L1:  
331     MOV DX, 0  
332     DIV BX  
333     PUSH DX  
334     INC CX  
335     INC DigitCount  
336     OR AX, AX  
337     JNE PN_L1  
338  
339 PN_L2:  
340     POP DX  
341     ADD DL, 48  
342     MOV AH, 02H  
343     INT 21H  
344     LOOP PN_L2  
345  
346     POP DX  
347     POP CX  
348     POP BX  
349     POP AX  
350     RET  
351 PRINT_NUMBER ENDP  
352 END MAIN
```

2.4 CONTROL FLOW (LOOPS AND DECISIONS)

ITEM SELECTION LOOP:

Displays current total

- Prompts user for selection (1-5 for items, 6 for invoice)
- Uses CMP and JE (jump if equal) to route to the appropriate handler
- Continues loop until user selects option 6

INVOICE ITEM DISPLAY LOGIC:

For each item:

- CMP Count_Item, 0 (check if purchased)
- JE SKIP_Item (skip if not purchased)
- Prints the name of the item, quantity, price, and total amount of the item purchased.
- Calculate and print item total (qty × price)
- Continue to the next item

DISCOUNT DECISION LOGIC:

Check thresholds from highest to lowest:

- If Grand_Total >= 10000: Apply 50% discount
- Else if >= 6000: Apply 30% discount
- Else if >= 4000: Apply 20% discount
- Else if >= 2000: Apply 10% discount
- Else: No discount

```
220 APPLY_50:
221     PRINT Congrats_50
222     MOV BX, 50
223     JMP DO_MATH
224 APPLY_30:
225     PRINT Congrats_30
226     MOV BX, 30
227     JMP DO_MATH
228 APPLY_20:
229     PRINT Congrats_20
230     MOV BX, 20
231     JMP DO_MATH
232 APPLY_10:
233     PRINT Congrats_10
234     MOV BX, 10
235
236 DO_MATH:
237     MUL BX
238     MOV CX, 100
239     DIV CX
240     MOV BX, AX
241
242     PRINT Discount_Prefix
243     MOV AX, BX
244     CALL PRINT_NUMBER
245
246     SUB Grand_Total, BX
```

3. SOURCE CODE

```
0001 .MODEL SMALL
0002 .STACK 100H
0003
0004 .DATA
0005
0006 ;UI Messages
0007 Welcome_Msg DB 10,13,'WELCOME TO JUMP IF HUNGRY <JZ> MART$'
0008 Menu_Header DB 10,13,10,13,'----- ITEMS MENU -----$',10,13
0009 Item_Menu_1 DB 10,13,'1. VEGETABLES - 80 RS$'
0010 Item_Menu_2 DB 10,13,'2. EGGS <DOZEN> - 300 RS$'
0011 Item_Menu_3 DB 10,13,'3. CANDIES - 5 RS$'
0012 Item_Menu_4 DB 10,13,'4. DAIRY PRODUCTS - 250 RS$'
0013 Item_Menu_5 DB 10,13,'5. FRUITS - 150 RS$'
0014
0015 Status_Msg DB 10,13,10,13,'CURRENT TOTAL: RS $'
0016 Choice_Msg DB 10,13,'SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: $'
0017
0018 ;Quantity Prompts
0019 Prompt_Veg DB 10,13,'HOW MANY VEGETABLES DO YOU WANT TO PURCHASE? $'
0020 Prompt_Eggs DB 10,13,'HOW MANY DOZEN EGGS DO YOU WANT TO PURCHASE? $'
0021 Prompt_Candy DB 10,13,'HOW MANY CANDIES DO YOU WANT TO PURCHASE? $'
0022 Prompt_Dairy DB 10,13,'HOW MANY DAIRY PRODUCTS DO YOU WANT TO PURCHASE? $'
0023 Prompt_Fruits DB 10,13,'HOW MANY FRUITS DO YOU WANT TO PURCHASE? $'
0024
0025 ;Invoice Header
0026 Invoice_Title DB 10,13,10,13,'===== FINAL INVOICE =====$'
0027 Column_Header DB 10,13,'ITEM NAME QTY PRICE ITEM TOTAL$'
0028 Separator_Line DB 10,13,'-----$'
0029
0030 ;Item Name Strings
0031 Name_Veg DB 10,13,'Vegetables $'
0032 Name_Eggs DB 10,13,'Eggs $'
0033 Name_Candy DB 10,13,'Candy $'
0034 Name_Dairy DB 10,13,'Dairy $'
0035 Name_Fruits DB 10,13,'Fruits $'
0036
0037 ;Congratulations Messages
0038 Sub_Total_Msg DB 10,13,10,13,'SUBTOTAL: RS $'
0039 Congrats_10 DB 10,13,'CONGRATULATIONS! YOU GOT A 10% DISCOUNT!$'
0040 Congrats_20 DB 10,13,'CONGRATULATIONS! YOU GOT A 20% DISCOUNT!$'
0041 Congrats_30 DB 10,13,'CONGRATULATIONS! YOU GOT A 30% DISCOUNT!$'
0042 Congrats_50 DB 10,13,'CONGRATULATIONS! YOU GOT A 50% DISCOUNT!$'
0043 Discount_Prefix DB 10,13,'BONUS DISCOUNT: RS $'
0044 Grand_Total_Msg DB 10,13,'GRAND TOTAL: RS $'
0045 Exit_Msg DB 10,13,10,13,'THANK YOU FOR SHOPPING AT JZ MART!$'
0046
0047 ;Arrays
0048 ;Price array <index 0=Veg, 1=Eggs, 2=Candy, 3=Dairy, 4=Fruits>
0049 Prices DW 80, 300, 5, 250, 150
0050
0051 ;Quantity array <parallel to Prices>
0052 Counts DW 0, 0, 0, 0, 0
0053
0054 ;Name pointer array <stores offsets of each Name_xx string>
0055 NamePtrs DW Name_Veg, Name_Eggs, Name_Candy, Name_Dairy, Name_Fruits
0056
0057 ;Prompt pointer array <stores offsets of each Prompt_xx string>
0058 PromptPtrs DW Prompt_Veg, Prompt_Eggs, Prompt_Candy, Prompt_Dairy, Prompt_Fruits
0059
0060 ;Variables
0061 Grand_Total DW 0
0062 DigitCount DW 0
0063
0064 ;Macros
0065 PRINT MACRO MSG
0066 LEA DX, MSG
0067 MOV AH, 09H
0068 INT 21H
0069 ENDM
```

```

071 .CODE
072 MAIN PROC
073     MOV AX, @DATA
074     MOV DS, AX
075
076     ; Display welcome and menu
077     PRINT Welcome_Msg
078     PRINT Menu_Header
079     PRINT Item_Menu_1
080     PRINT Item_Menu_2
081     PRINT Item_Menu_3
082     PRINT Item_Menu_4
083     PRINT Item_Menu_5
084
085     ;-----
086     ; ITEM SELECTION LOOP
087     ; Displays current total, prompts user for selection
088     ; Uses CMP/JE to route to index handler
089     ;-----
090     ITEM_SELECTION_LOOP:
091         PRINT Status_Msg
092         MOV AX, Grand_Total
093         CALL PRINT_NUMBER
094         PRINT Choice_Msg
095
096         MOV AH, 01H
097         INT 21H
098         SUB AL, 48
099
100         CMP AL, 6
101         JE GENERATE_INVOICE
102         CMP AL, 1
103         JE IDX_0
104         CMP AL, 2
105         JE IDX_1
106         CMP AL, 3
107         JE IDX_2
108         CMP AL, 4
109         JE IDX_3
110         CMP AL, 5
111         JE IDX_4
112         JMP ITEM_SELECTION_LOOP
113
114     IDX_0: MOV SI, 0
115           JMP BUY_ITEM
116     IDX_1: MOV SI, 2
117           JMP BUY_ITEM
118     IDX_2: MOV SI, 4
119           JMP BUY_ITEM
120     IDX_3: MOV SI, 6
121           JMP BUY_ITEM
122     IDX_4: MOV SI, 8
123           JMP BUY_ITEM
124
125     ;-----
126     ; BUY_ITEM PROCEDURE (Single handler replacing 5 separate ones)
127     ; SI = byte offset into Prices[], Counts[], NamePtrs[], PromptPtrs[]
128     ;-----
129     BUY_ITEM:
130         MOV BX, PromptPtrs[SI]
131         MOV DX, BX
132         MOV AH, 09H
133         INT 21H
134
135         CALL GET_QUANTITY
136
137         ADD Counts[SI], AX
138
139         MOV BX, Prices[SI]
140         MUL BX
141         ADD Grand_Total, AX
142
143         JMP ITEM_SELECTION_LOOP

```

```

145 ;-----
146 ; GENERATE INVOICE PROCEDURE
147 ; Iterates through arrays using LOOP + SI index
148 ;-----
149 GENERATE_INVOICE:
150     MOV AX, 03H
151     INT 10H
152
153     PRINT Invoice_Title
154     PRINT Column_Header
155     PRINT Separator_Line
156
157     MOV SI, 0
158     MOV CX, 5

160 INVOICE_LOOP:
161     CMP Counts[SI], 0
162     JE SKIP_ITEM
163
164     ;Prints the item name
165     MOV BX, NamePtrs[SI]
166     MOV DX, BX
167     MOV AH, 09H
168     INT 21H
169
170     ;Prints the quantity
171     MOV AX, Counts[SI]
172     CALL PRINT_NUMBER
173     PUSH CX
174     MOV CX, 8
175     SUB CX, DigitCount
176     CALL PRINT_SPACES
177     POP CX
178
179     ;Prints the unit price
180     MOV AX, Prices[SI]
181     CALL PRINT_NUMBER
182     PUSH CX
183     MOV CX, 10
184     SUB CX, DigitCount
185     CALL PRINT_SPACES
186     POP CX
187
188     ;Calculates and print the item total
189     MOV AX, Counts[SI]
190     MOV BX, Prices[SI]
191     MUL BX
192     CALL PRINT_NUMBER

194 SKIP_ITEM:
195     ADD SI, 2
196     LOOP INVOICE_LOOP
197
198     PRINT Separator_Line

```

```

200 ;-----
201 ; FINANCIAL SUMMARY AND DISCOUNT LOGIC
202 ;-----
203     PRINT Sub_Total_Msg
204     MOV AX, Grand_Total
205     CALL PRINT_NUMBER
206
207     MOV AX, Grand_Total
208
209     ;Types of discount applied based on total amount of purchases
210     CMP AX, 10000
211     JAE APPLY_50
212     CMP AX, 6000
213     JAE APPLY_30
214     CMP AX, 4000
215     JAE APPLY_20
216     CMP AX, 2000
217     JAE APPLY_10
218     JMP DISPLAY_FINAL
219
220 APPLY_50:
221     PRINT Congrats_50
222     MOV BX, 50
223     JMP DO_MATH
224
225 APPLY_30:
226     PRINT Congrats_30
227     MOV BX, 30
228     JMP DO_MATH
229
230 APPLY_20:
231     PRINT Congrats_20
232     MOV BX, 20
233     JMP DO_MATH
234
235 APPLY_10:
236     PRINT Congrats_10
237     MOV BX, 10
238
239 DO_MATH:
240     MUL BX
241     MOV CX, 100
242     DIV CX
243     MOV BX, AX
244
245     PRINT Discount_Prefix
246     MOV AX, BX
247     CALL PRINT_NUMBER
248
249     SUB Grand_Total, BX
250
251 DISPLAY_FINAL:
252     PRINT Grand_Total_Msg
253     MOV AX, Grand_Total
254     CALL PRINT_NUMBER
255
256     PRINT Exit_Msg
257
258     MOV AH, 4CH
259     INT 21H
260
261 MAIN ENDP

```

```

259 ;=====
260 ; HELPER PROCEDURES
261 ;=====
262
263 ;-----
264 ; GET_QUANTITY: Reads multi-digit integer from keyboard
265 ; Returns: integer value in AX
266 ; Algorithm: accumulate-and-shift (BX = BX*10 + digit)
267 ;-----
268 GET_QUANTITY PROC
269     PUSH BX
270     PUSH CX
271     MOV BX, 0
272     MOV CX, 10
273
274 GET_Q_LOOP:
275     MOV AH, 01H
276     INT 21H
277     CMP AL, 13
278     JE GET_Q_DONE
279     SUB AL, 48
280     MOV AH, 0
281     PUSH AX
282     MOV AX, BX
283     MUL CX
284     MOV BX, AX
285     POP AX
286     ADD BX, AX
287     JMP GET_Q_LOOP
288
289 GET_Q_DONE:
290     MOV AX, BX
291     POP CX
292     POP BX
293     RET
294 GET_QUANTITY ENDP

```

```

296 ;-----
297 ; PRINT_SPACES: Prints CX spaces for column alignment
298 ; Receives: CX = number of spaces
299 ;-----
300 PRINT_SPACES PROC
301     CMP CX, 0
302     JLE PS_END
303     PUSH AX
304     PUSH DX
305     MOV AH, 02H
306     MOV DL, ' '
307 PS_L:
308     INT 21H
309     LOOP PS_L
310     POP DX
311     POP AX
312 PS_END:
313     RET
314 PRINT_SPACES ENDP

```

```

316 ;-----
317 ; PRINT_NUMBER: Converts 16-bit unsigned int in AX to ASCII and displays
318 ; Sets DigitCount for use by PRINT_SPACES alignment
319 ; Algorithm: divide by 10, push remainders, pop in order
320 ;-----
321 PRINT_NUMBER PROC
322     PUSH AX
323     PUSH BX
324     PUSH CX
325     PUSH DX
326     MOV DigitCount, 0
327     MOV BX, 10
328     MOV CX, 0
329
330 PN_L1:
331     MOV DX, 0
332     DIV BX
333     PUSH DX
334     INC CX
335     INC DigitCount
336     OR AX, AX
337     JNE PN_L1
338
339 PN_L2:
340     POP DX
341     ADD DL, 48
342     MOV AH, 02H
343     INT 21H
344     LOOP PN_L2
345
346     POP DX
347     POP CX
348     POP BX
349     POP AX
350     RET
351 PRINT_NUMBER ENDP
352 END MAIN

```

4. PROGRAM OUTPUT

Output Sequence:

[1] WELCOME AND MENU DISPLAY:

```
WELCOME TO JUMP IF HUNGRY <JZ> MART

----- ITEMS MENU -----
1. VEGETABLES      - 80 RS
2. EGGS <DOZEN>    - 300 RS
3. CANDIES         - 5 RS
4. DAIRY PRODUCTS - 250 RS
5. FRUITS          - 150 RS

CURRENT TOTAL: RS 0
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: _
```

[2] ITEM SELECTION AND QUANTITY INPUT:

User input: 1 (for vegetables)

```
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: 1
HOW MANY VEGETABLES DO YOU WANT TO PURCHASE? 10

CURRENT TOTAL: RS 800
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: _
```

[3] MULTIPLE ITEM PURCHASES:

```
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: 1
HOW MANY VEGETABLES DO YOU WANT TO PURCHASE? 10

CURRENT TOTAL: RS 800
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: 2
HOW MANY DOZEN EGGS DO YOU WANT TO PURCHASE? 2

CURRENT TOTAL: RS 1400
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: 3
HOW MANY CANDIES DO YOU WANT TO PURCHASE? 50

CURRENT TOTAL: RS 1650
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE: 4
HOW MANY DAIRY PRODUCTS DO YOU WANT TO PURCHASE? 2

CURRENT TOTAL: RS 2150
SELECT ITEM <1-5> OR 6 TO GENERATE INVOICE:
```


[4] FINAL INVOICE OUTPUT (After selecting option 6):

===== FINAL INVOICE =====			
ITEM NAME	QTY	PRICE	ITEM TOTAL
Vegetables	10	80	800
Eggs	2	300	600
Candy	50	5	250
Dairy	2	250	500

SUBTOTAL: RS 2150			
CONGRATULATIONS! YOU GOT A 10% DISCOUNT!			
BONUS DISCOUNT: RS 234			
GRAND TOTAL: RS 1916			
THANK YOU FOR SHOPPING AT JZ MART!			

KEY FEATURES DEMONSTRATED:

1. Menu Display:

- Welcome banner displayed
- All the items are shown correctly and clearly
- The menu display is beautiful through spacing and has an overall nice design.

2. Item Selection:

- The user can select items from the 5 items stated above.
- The user can reselect the items many times.
- Total updates after every item purchased.

3. Quantity Input:

- The user can enter the quantity of the item to be purchased.
- By pressing the enter key, the flow of the loop resets to the first message.
- On entering the quantity, the system automatically multiplies it with the amount of one item and add it to the total amount.

4. Invoice Formatting:

- Formatted nicely.
- Purchased items are displayed in the Invoice.
- The quantity of the items purchased, their prices, and the total of each purchase are displayed in separate columns.

5. Discount Logic:

- Discount applied based on the total amount as mentioned in the PDF.
- Discounts are applied according to the slabs mentioned in the PDF.
- On every discount achieved, a congratulatory message is displayed on the screen.
- The discounted amount is shown and applied.

5. CONCLUSION

REQUIREMENTS MET:

- ✓ Item Storage: 5 items stored in memory with names and prices
- ✓ User Interaction: Menu-driven interface with item selection and quantity input
- ✓ Billing Logic: Accurate calculation of item and grand totals
- ✓ Invoice Generation: Professionally formatted output with all details
- ✓ Bonus Feature: Tiered discount system (10%, 20%, 30%, 50%)
- ✓ Implemented the use of arrays and macro features into the code.

TECHNICAL REQUIREMENTS:

- ✓ 6 independent procedures with clear responsibilities
- ✓ **Use of MACROS:** PRINT macro reduces code duplication
- ✓ **Data Structures:** Proper use of arrays and variables. DATA segment
- ✓ **Control Flow:** Loops and decisions with CMP/JE/JAE instructions
- ✓ **Code Quality:** Well-commented, clear labels, readable structure

PROCEDURES IMPLEMENTED:

- MAIN - Entry point, menu display, selection loop management
- GET_QUANTITY - Multi-digit integer parsing from keyboard
- PRINT_NUMBER - 16-bit to ASCII conversion and display
- PRINT_SPACES - Column alignment with dynamic spacing
- PRINT_MACRO - String output encapsulation

CHALLENGES AND SOLUTIONS:

Challenge 1: Multi-Digit Input Parsing

Problem: DOS only provides single-character input

Solution: Implemented GET_QUANTITY using accumulate-and-shift algorithm

Result: Supports arbitrary quantity values

Challenge 2: Dynamic Number-to-ASCII Conversion

Problem: Need to display variable-width numbers in columns

Solution: PRINT_NUMBER procedure using division-based digit extraction

Result: Accurate decimal output with digit counting for alignment

Challenge 3: Invoice Column Alignment

Problem: A number of different widths misalign columns

Solution: Track DigitCount in PRINT_NUMBER, use PRINT_SPACES for padding

Result: Professional, aligned invoice formatting

Challenge 4: Tiered Discount Logic

Problem: Different discount percentages based on the total amount

Solution: Cascading threshold checks (highest to lowest)

Result: Correct tier selection with minimal code

Challenge 5: Running Total Display

Problem: Must update and display the total after each purchase

Solution: Accumulate in the Grand_Total variable, display before each selection

Result: User can track running cost in real-time

LEARNING OUTCOMES DEMONSTRATED:

- ✓ **8086 Assembly Language Mastery:** Instruction set, registers, memory addressing
- ✓ **Procedural Programming:** Procedure design, parameter passing, stack preservation
- ✓ **Macros and Code Reuse:** Macro definition and usage, reducing repetition
- ✓ **Real-World Problem Solving:** Business logic to assembly translation
- ✓ **DOS Programming:** Interrupt-driven I/O and console operations
- ✓ **Numerical Algorithms:** Number conversion, percentage calculation, arithmetic

Department of Computer Science & Information Technology
CS-252 Computer Architecture & Organization
Complex Computing Activity (CCA) through Open-Ended Lab (OEL)

Student Name: _____ Student Roll No.: _____

Criteria and Scales		
Fully Complete & Functional (2)	Partially Complete (1)	Incomplete/Incorrect (0)
Criterion 1: Use of Arrays and Memory (Aligned CA Attributes: CA-1, CA-5) To what extent has the student correctly used memory locations and arrays to store and manage item data such as names, prices, quantities, and totals?		
Arrays and memory are correctly used with proper indexing.	Arrays are partially implemented or incorrectly indexed.	Arrays or memory usage are missing or wrong.
Criterion 2: Loops, Control Flow and Modular Design (Aligned CA Attributes: CA-2, CA-5) To what extent does the program use loops, conditional jumps, and well-defined procedures to control flow, manage repetition, and enhance modularity?		
Loops, conditionals, and procedures correctly control program flow, manage repetition, and enhance modularity.	Some loops or procedures work but are incomplete, inconsistent, or partially functional.	Control flow or procedures are missing or incorrectly implemented.
Criterion 3: Macro Usage (Aligned CA Attributes: CA-1, CA-3) To what extent has the student defined and used macros to reduce code repetition and improve clarity?		
Macros are correctly defined and reduce repetition, improving code clarity.	Macros are used inconsistently or have minimal effect.	Macros are missing or incorrectly implemented.
Criterion 4: Invoice Generation & Computational Correctness (Aligned CA Attributes: CA-2, CA-3) To what extent has the student correctly implemented invoice generation logic, including accurate computation of item totals and the final bill amount?		
Invoice and totals are accurate and clearly displayed.	Invoice has minor calculation or formatting errors.	Invoice is missing or contains major errors.
Criterion 5: Documentation and Explanation (Aligned CA Attributes: CA-1, CA-5) To what extent has the student documented and explained the system design, program logic, code structure, and output results?		
Documentation clearly explains design, logic, code, and output with relevant screenshots.	Documentation exists but is unclear or incomplete.	Documentation is missing or poorly explained.

Total Marks: ____/10 ____

Teacher's Signature: _____

Department of Computer Science & Information Technology
CS-252 Computer Architecture & Organization
Complex Computing Activity (CCA) through Open-Ended Lab (OEL)

Student Name: _____ Student Roll No.: _____

Criteria and Scales		
Fully Complete & Functional (2)	Partially Complete (1)	Incomplete/Incorrect (0)
Criterion 1: Use of Arrays and Memory (Aligned CA Attributes: CA-1, CA-5) To what extent has the student correctly used memory locations and arrays to store and manage item data such as names, prices, quantities, and totals?		
Arrays and memory are correctly used with proper indexing.	Arrays are partially implemented or incorrectly indexed.	Arrays or memory usage are missing or wrong.
Criterion 2: Loops, Control Flow and Modular Design (Aligned CA Attributes: CA-2, CA-5) To what extent does the program use loops, conditional jumps, and well-defined procedures to control flow, manage repetition, and enhance modularity?		
Loops, conditionals, and procedures correctly control program flow, manage repetition, and enhance modularity.	Some loops or procedures work but are incomplete, inconsistent, or partially functional.	Control flow or procedures are missing or incorrectly implemented.
Criterion 3: Macro Usage (Aligned CA Attributes: CA-1, CA-3) To what extent has the student defined and used macros to reduce code repetition and improve clarity?		
Macros are correctly defined and reduce repetition, improving code clarity.	Macros are used inconsistently or have minimal effect.	Macros are missing or incorrectly implemented.
Criterion 4: Invoice Generation & Computational Correctness (Aligned CA Attributes: CA-2, CA-3) To what extent has the student correctly implemented invoice generation logic, including accurate computation of item totals and the final bill amount?		
Invoice and totals are accurate and clearly displayed.	Invoice has minor calculation or formatting errors.	Invoice is missing or contains major errors.
Criterion 5: Documentation and Explanation (Aligned CA Attributes: CA-1, CA-5) To what extent has the student documented and explained the system design, program logic, code structure, and output results?		
Documentation clearly explains design, logic, code, and output with relevant screenshots.	Documentation exists but is unclear or incomplete.	Documentation is missing or poorly explained.

Total Marks: ____/10 ____

Teacher's Signature: _____

Department of Computer Science & Information Technology
CS-252 Computer Architecture & Organization
Complex Computing Activity (CCA) through Open-Ended Lab (OEL)

Student Name: _____ Student Roll No.: _____

Criteria and Scales		
Fully Complete & Functional (2)	Partially Complete (1)	Incomplete/Incorrect (0)
Criterion 1: Use of Arrays and Memory (Aligned CA Attributes: CA-1, CA-5) To what extent has the student correctly used memory locations and arrays to store and manage item data such as names, prices, quantities, and totals?		
Arrays and memory are correctly used with proper indexing.	Arrays are partially implemented or incorrectly indexed.	Arrays or memory usage are missing or wrong.
Criterion 2: Loops, Control Flow and Modular Design (Aligned CA Attributes: CA-2, CA-5) To what extent does the program use loops, conditional jumps, and well-defined procedures to control flow, manage repetition, and enhance modularity?		
Loops, conditionals, and procedures correctly control program flow, manage repetition, and enhance modularity.	Some loops or procedures work but are incomplete, inconsistent, or partially functional.	Control flow or procedures are missing or incorrectly implemented.
Criterion 3: Macro Usage (Aligned CA Attributes: CA-1, CA-3) To what extent has the student defined and used macros to reduce code repetition and improve clarity?		
Macros are correctly defined and reduce repetition, improving code clarity.	Macros are used inconsistently or have minimal effect.	Macros are missing or incorrectly implemented.
Criterion 4: Invoice Generation & Computational Correctness (Aligned CA Attributes: CA-2, CA-3) To what extent has the student correctly implemented invoice generation logic, including accurate computation of item totals and the final bill amount?		
Invoice and totals are accurate and clearly displayed.	Invoice has minor calculation or formatting errors.	Invoice is missing or contains major errors.
Criterion 5: Documentation and Explanation (Aligned CA Attributes: CA-1, CA-5) To what extent has the student documented and explained the system design, program logic, code structure, and output results?		
Documentation clearly explains design, logic, code, and output with relevant screenshots.	Documentation exists but is unclear or incomplete.	Documentation is missing or poorly explained.

Total Marks: _____/10 _____

Teacher's Signature: _____